

Operating Systems

BCA — Semester 4 — 2019 — Answer Sheet

Subject Code: CACS251

Group B

Attempt any SIX questions. [6×5=30]

2. Explain the functions of operating system.

[5]

Answer: Functions of Operating System:

- 1. Process Management:** The OS manages the creation, scheduling, and termination of processes. It allocates CPU time using scheduling algorithms (FCFS, SJF, Round Robin, etc.) and handles inter-process communication and synchronization.
- 2. Memory Management:** The OS keeps track of memory usage, allocates memory to processes, and deallocates it when no longer needed. It manages virtual memory, paging, segmentation, and swapping to optimize RAM usage.
- 3. File Management:** The OS provides a file system to store, retrieve, and organize data on storage devices. It handles file creation, deletion, naming, directory structures, permissions, and disk space allocation.
- 4. Device Management:** The OS manages device communication via device drivers. It handles I/O operations, buffer caching, spooling, and ensures efficient use of peripherals (printers, disks, keyboards).
- 5. Security and Access Control:** The OS protects system resources and user data through authentication, authorization, encryption, and access control lists (ACL). It prevents unauthorized access and ensures user privacy.
- 6. User Interface:** The OS provides interfaces for user interaction — Command Line Interface (CLI) and Graphical User Interface (GUI). It abstracts hardware complexity, making the system user-friendly.
- 7. Error Detection and Handling:** The OS constantly monitors the system for errors (hardware failures, software crashes) and takes corrective actions to maintain stability and reliability.

3. How does semaphore help in dining philosopher problem?

[5]

Answer: Dining Philosopher Problem:

Five philosophers sit around a table with five chopsticks (forks) between them. Each philosopher needs two chopsticks to eat. The problem is to design a solution where no philosopher starves (deadlock-free) and maximum parallelism is achieved.

Role of Semaphore:

A semaphore is a synchronization variable that controls access to shared resources. In the dining philosopher problem, each chopstick is represented by a binary semaphore (initialized to 1).

Algorithm:

- Philosopher i picks left chopstick by executing **wait(chopstick[i])**
- Philosopher i picks right chopstick by executing **wait(chopstick[(i+1)%5])**
- After eating, philosopher releases both chopsticks using **signal(chopstick[i])** and **signal(chopstick[(i+1)%5])**

Deadlock Issue:

If all philosophers pick up their left chopstick simultaneously, they all wait for the right chopstick — deadlock.

Solution using Semaphore:

1. **Limit philosophers:** Allow at most 4 philosophers to sit simultaneously using a counting semaphore.
2. **Asymmetric solution:** Odd philosophers pick left first, even philosophers pick right first.
3. **Monitor or mutex:** Protect the pickup and putdown operations with a mutex so that picking up both forks is an atomic operation.

Semaphores ensure mutual exclusion and prevent race conditions, providing a structured way to solve synchronization problems.

4. Consider a system with three resources. Each process needs a maximum of two resources. Prove that deadlock cannot occur. Explain with answer.

[5]

Answer: Given:

- Total resources (R) = 3 (of the same type)
- Maximum need of each process = 2 resources
- Number of processes can be any

Proof that Deadlock Cannot Occur:

For deadlock to occur, the four Coffman conditions must hold simultaneously: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait.

Consider the **worst-case scenario** for deadlock: each process holds one resource and waits for another.

If there are **3 processes**, each holding 1 resource, then all 3 resources are allocated. Each process needs 1 more. But since all resources are held, no process can proceed — this seems like deadlock.

However, the key insight is: **if there are only 2 processes**, each can hold at most 1 resource (total 2 allocated), leaving 1 resource free. The free resource can be allocated to either process, allowing it to complete and release both resources.

If there are **3 or more processes**, at least one process must be able to obtain 2 resources because the maximum need is 2 and there are 3 resources total. The system can always satisfy at least one process completely, which then releases its resources for others.

General Proof:

Let n = number of processes, $R = 3$, $\text{Max} = 2$.

If each process holds $(\text{Max} - 1) = 1$ resource, total held = n .

For deadlock, we need $n \geq R$ (all resources held). So $n \geq 3$.

But if $n = 2$, total held = 2, leaving 1 free. One process gets the free resource, finishes, and releases 2 resources.

If $n \geq 3$, the pigeonhole principle ensures at least one process can get its full requirement (2) because not all processes can simultaneously hold exactly 1 and block — the OS can allocate the third resource to any waiting process.

Conclusion: Since the maximum need (2) is less than the total resources (3), the system can always find at least one process that can be satisfied completely, preventing deadlock.

5. If processes arrive at an average of six processes per minute and each requires 8 seconds of service time. Estimate the fraction of time the CPU is busy. [M/M/1 queue problem]

[5]

Answer: Given (M/M/1 Queue):

- Arrival rate (λ) = 6 processes per minute = $6/60 = 0.1$ processes/second
- Service time = 8 seconds per process
- Service rate (μ) = $1 / 8 = 0.125$ processes/second

Traffic Intensity (ρ):

$$\rho = \lambda / \mu = 0.1 / 0.125 = 0.8$$

Interpretation:

Traffic intensity ρ represents the fraction of time the CPU is busy (utilization).

Fraction of time CPU is busy = $\rho = 0.8 = 80\%$

Additional Metrics (for completeness):

- Average number of processes in system (L) = $\rho / (1 - \rho) = 0.8 / 0.2 = 4$ processes
- Average time in system (W) = $1 / (\mu - \lambda) = 1 / 0.025 = 40$ seconds
- Average number in queue (L_q) = $\rho^2 / (1 - \rho) = 0.64 / 0.2 = 3.2$ processes

Conclusion: The CPU is busy 80% of the time and idle 20% of the time.

6. Consider the page reference string: 8,2,3,8,3. A program has three page frames for each of the processes. Determine which pages are replaced using FIFO page replacement algorithm.

[5]

Answer: Given:

- Page reference string: 8, 2, 3, 8, 3
- Number of page frames: 3
- Algorithm: FIFO (First In First Out)

Page Replacement Trace:

Reference 8 2 3 8 3 Frame 1 8 8 8 8 Frame 2 2 2 2 2 Frame 3 -- 3 3 3 Page Fault Yes Yes Yes No No

* indicates page loaded into memory.

Step-by-step:

1. **Reference 8:** Memory is empty. Load 8 into Frame 1. **Page Fault**
2. **Reference 2:** 2 not in memory. Load 2 into Frame 2. **Page Fault**
3. **Reference 3:** 3 not in memory. Load 3 into Frame 3. **Page Fault**
4. **Reference 8:** 8 is already in Frame 1. **No Page Fault**
5. **Reference 3:** 3 is already in Frame 3. **No Page Fault**

Result:

- Total page faults = 3
- Total hits = 2
- No pages are replaced because the reference string contains only 3 unique pages and we have exactly 3 frames. All pages fit into memory without needing replacement.

Note: If the reference string had more unique pages than frames, the oldest page (the one at the front of the FIFO queue) would be replaced.

7. What is authentication? How worms are differing from viruses?

[5]

Answer: Authentication:

Authentication is the process of verifying the identity of a user, device, or system. It ensures that the entity requesting access is who it claims to be. Common authentication methods include:

1. **Password-based:** User provides a secret string known only to them.
2. **Biometric:** Uses physical characteristics (fingerprint, iris, face recognition).
3. **Token-based:** Uses hardware or software tokens (OTP, smart cards).
4. **Multi-factor Authentication (MFA):** Combines two or more methods (password + OTP).

Difference between Worms and Viruses:

Aspect	Virus	Worm
Definition	A malicious program that attaches itself to a host file or program and requires human action to spread.	A standalone malicious program that replicates itself and spreads automatically across networks without host attachment.
Host Requirement	Requires a host file or program to execute.	Does not require a host file; runs independently.
Spreading Mechanism	Spreads when the infected file is shared or executed by the user.	Spreads automatically over networks by exploiting vulnerabilities.
Human Intervention	Usually requires human action (opening a file) to propagate.	No human intervention needed; self-propagating.
Speed	Slow spreading.	Rapid spreading across networks.
Payload	May corrupt, delete, or alter files.	Consumes bandwidth and system resources; may carry payloads.
Examples	LOVEYOU, Melissa, Chernobyl (CIH).	Morris Worm, Conficker, Stuxnet, WannaCry.

Group C

Attempt any TWO questions. [2×10=20]

8. Explain process scheduling algorithms with examples.

[10]

Answer: Process Scheduling Algorithms:

Process scheduling is the mechanism used by the OS to allocate CPU time to processes. The goal is to maximize CPU utilization, throughput, and fairness while minimizing waiting and response time.

1. First Come First Serve (FCFS):

Processes are executed in the order of their arrival.

- **Example:** Processes P1 (burst 24), P2 (burst 3), P3 (burst 3) arrive in order.
- Gantt Chart: | P1 | P2 | P3 |
- Waiting times: P1=0, P2=24, P3=27. Average = $(0+24+27)/3 = 17$
- **Disadvantage:** Convoy effect — short processes wait behind long ones.

2. Shortest Job First (SJF):

The process with the smallest burst time is executed first.

- **Example:** Same processes. Order: P2, P3, P1.
- Waiting times: P2=0, P3=3, P1=6. Average = 3
- **Types:** Non-preemptive and Preemptive (Shortest Remaining Time First).

3. Priority Scheduling:

Each process has a priority. The highest priority process runs first.

- Can be preemptive or non-preemptive.
- **Problem:** Starvation of low-priority processes. **Solution:** Aging (gradually increasing priority of waiting processes).

4. Round Robin (RR):

Each process gets a small time quantum (e.g., 4 ms). If not finished, it goes to the tail of the ready queue.

- **Example:** Time quantum = 4. Processes P1(10), P2(3), P3(3).
- Gantt: P1 P2 P3 P1 P3 P1 P1
- Fair for time-sharing systems; no starvation.

5. Multilevel Queue Scheduling:

Ready queue is divided into multiple queues (e.g., foreground, background). Each queue has its own scheduling algorithm. Fixed priority between queues.

6. Multilevel Feedback Queue:

Allows processes to move between queues. I/O-bound and interactive processes get higher priority; CPU-bound processes move to lower-priority queues.

Comparison Table:

Algorithm	Type	Starvation	Overhead
FCFS	Non-preemptive	No	Low
SJF	Both	Yes	Medium
Priority	Both	Yes	Low
Round Robin	Preemptive	No	Medium
MLFQ	Preemptive	No	High

■ *Diagram Required:* Gantt charts should be drawn for each scheduling algorithm to illustrate process execution timelines.

9. Explain memory management techniques (paging and segmentation).

[10]

Answer: Memory Management Techniques:

Memory management is the functionality of an OS that handles primary memory. It keeps track of each byte in memory, which part is free or allocated, and how much is allocated.

1. Paging:

Paging is a fixed-size partitioning scheme that eliminates external fragmentation.

Concept:

- Physical memory is divided into fixed-size blocks called **frames**.
- Logical memory is divided into same-size blocks called **pages**.
- The OS maintains a **page table** for each process, mapping logical pages to physical frames.

Address Translation:

Logical address = (Page Number, Offset)

- Page Number indexes the page table to get Frame Number.
- Physical Address = (Frame Number, Offset)

Example:

Page size = 4 KB. Logical address space = 16 pages. Physical memory = 32 frames.

Logical address 0 maps to page 0 → frame 5 → physical address $5 \times 4096 + 0 = 20480$.

Advantages: No external fragmentation, simple memory allocation.

Disadvantages: Internal fragmentation (last page may not fill frame), page table overhead.

2. Segmentation:

Segmentation is a variable-size partitioning scheme that matches the logical structure of programs.

Concept:

- A program is divided into segments based on its logical units: code, data, stack, heap, etc.
- Each segment has a name and length.
- The OS maintains a **segment table** containing base (starting physical address) and limit (length) for each segment.

Address Translation:

Logical address = (Segment Number, Offset)

- Check if offset < limit. If not, trap (addressing error).
- Physical Address = Base + Offset

Example:

Segment 0 (Code): base=1000, limit=600

Segment 1 (Data): base=2000, limit=400

Logical address (1, 300) → physical address $2000 + 300 = 2300$.

Advantages: Matches program structure, supports sharing and protection per segment, no internal fragmentation.

Disadvantages: External fragmentation (variable-size holes), complex allocation.

Comparison:

Aspect	Paging	Segmentation	Block size	Fixed (pages)	Variable (segments)	Fragmentation	Internal	External	User awareness
Transparent	Visible to programmer	Table	Page table	Segment table	(base + limit)				
Sharing	Difficult	Easy (share whole segment)							

■ *Diagram Required:* Diagrams showing logical to physical address translation using page table and segment table.

10. Explain file system architecture and directory structures.

[10]

Answer: File System Architecture:

A file system is a method and data structure that the OS uses to control how data is stored and retrieved on storage devices.

Layers of File System:

- 1. Application Programs:** User-level programs that interact with files (text editors, browsers).
- 2. Logical File System:** Manages metadata (file control blocks, directory structures), handles security and protection, and provides the API for file operations (open, read, write, close).
- 3. File Organization Module:** Knows about files, their logical blocks, and physical blocks. Translates logical block addresses to physical block addresses and manages free space.
- 4. Basic File System (Physical I/O):** Issues generic commands to device drivers to read/write physical blocks on disk.
- 5. I/O Control (Device Drivers):** Low-level software that interacts directly with hardware controllers. Handles interrupts, manages device registers.
- 6. Devices:** Physical storage hardware (HDD, SSD, USB).

Directory Structures:

A directory is a special file that contains a list of files and subdirectories.

1. Single-Level Directory:

All files are in one directory.

- Simple but suffers from naming conflicts and poor organization.

2. Two-Level Directory:

Separate directory for each user.

- Path: /username/filename
- Reduces naming conflicts but no user cooperation.

3. Tree-Structured Directory:

A root directory with files and subdirectories forming a tree.

- Path: /home/user/docs/file.txt
- Supports efficient searching and grouping; most common structure.

4. Acyclic-Graph Directory:

Allows directories and files to be shared via links (symbolic/hard links).

- Same file can appear in multiple directories.
- No cycles allowed to avoid infinite loops.

5. General Graph Directory:

Allows cycles via links.

- Requires garbage collection or reference counting to reclaim disk space.
- More flexible but complex.

File System Implementation:

- **File Allocation Table (FAT):** Linked list of blocks stored in a table.
- **Inode (Index Node):** UNIX-style structure storing metadata and pointers to data blocks.
- **Extents:** Contiguous blocks allocated together for large files.

Diagram Description: A layered architecture diagram with Application at top, Logical FS, File Organization Module, Basic File System, I/O Control, and Devices at bottom.

■ *Diagram Required:* Layered file system architecture diagram and tree directory structure diagram.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.